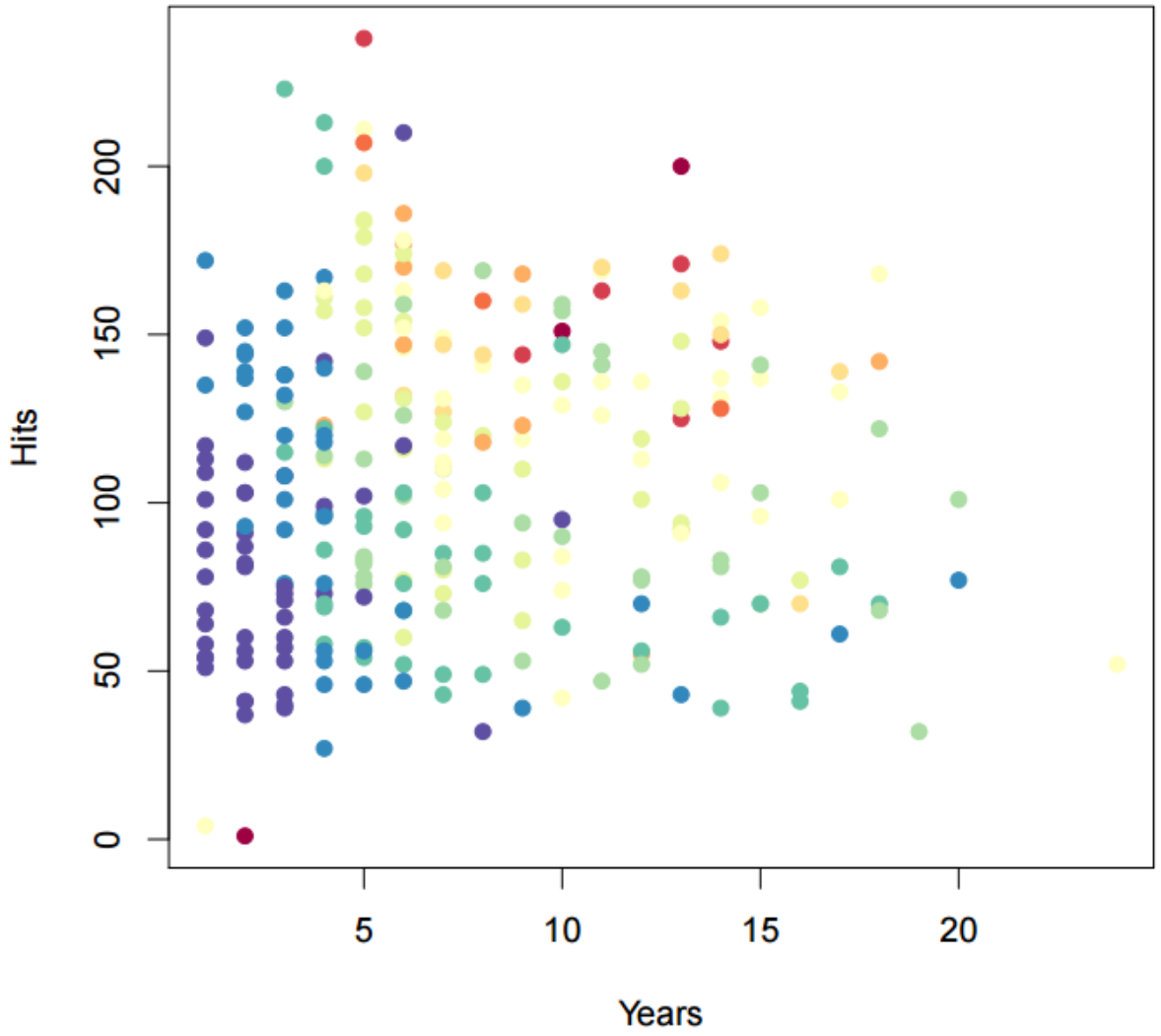
Machine Learning

CSE 343/543

Non Metric Classification – Decision Trees

Slide Credit: David Sontag, NYU; Tom Mitchell, CMU

Baseball Salary Data



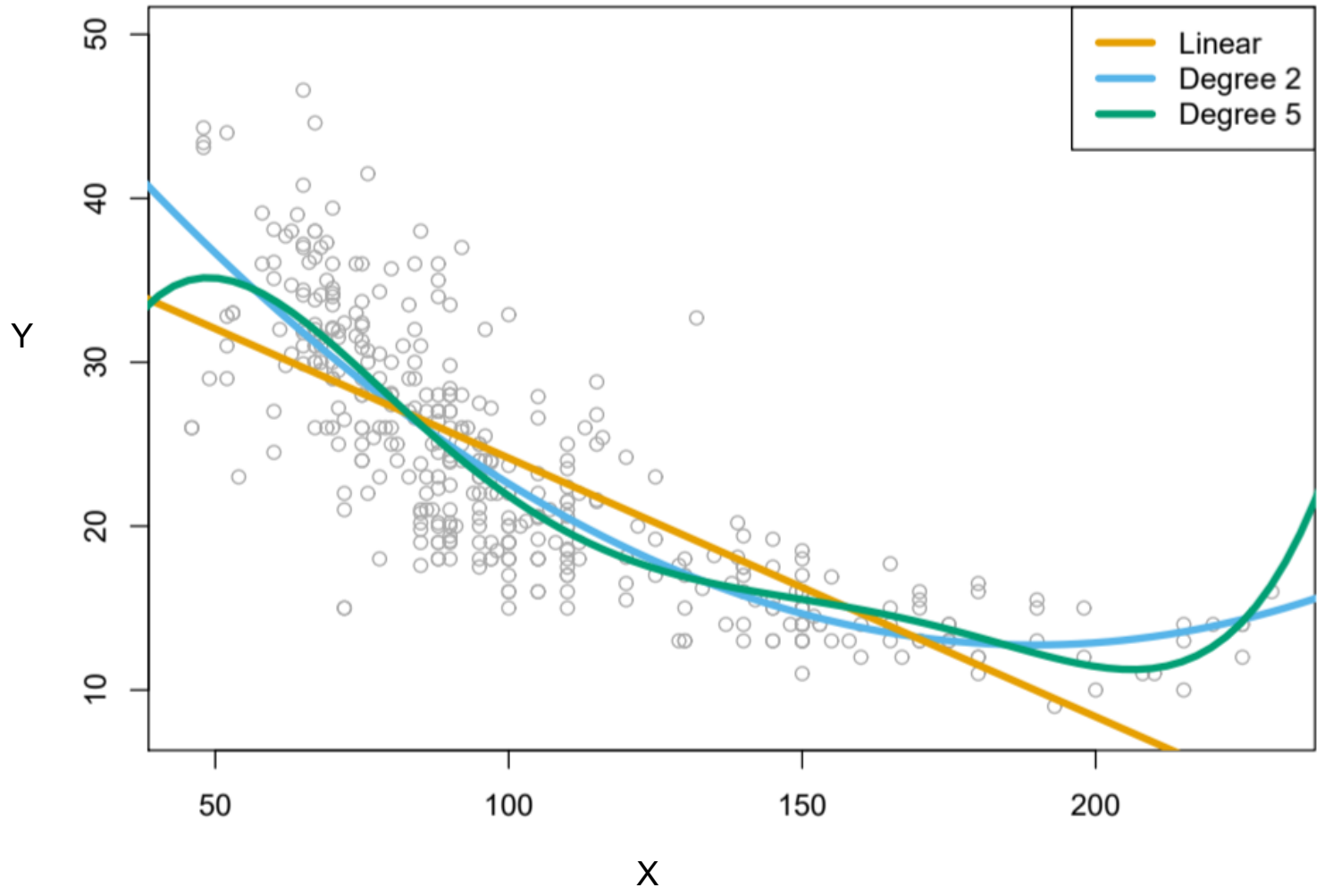
Red / Yellow → High
Blue / Green → Low

Recall: Linear Regression

- Least Squares regression

$$\min \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

- Could be used to fit polynomials



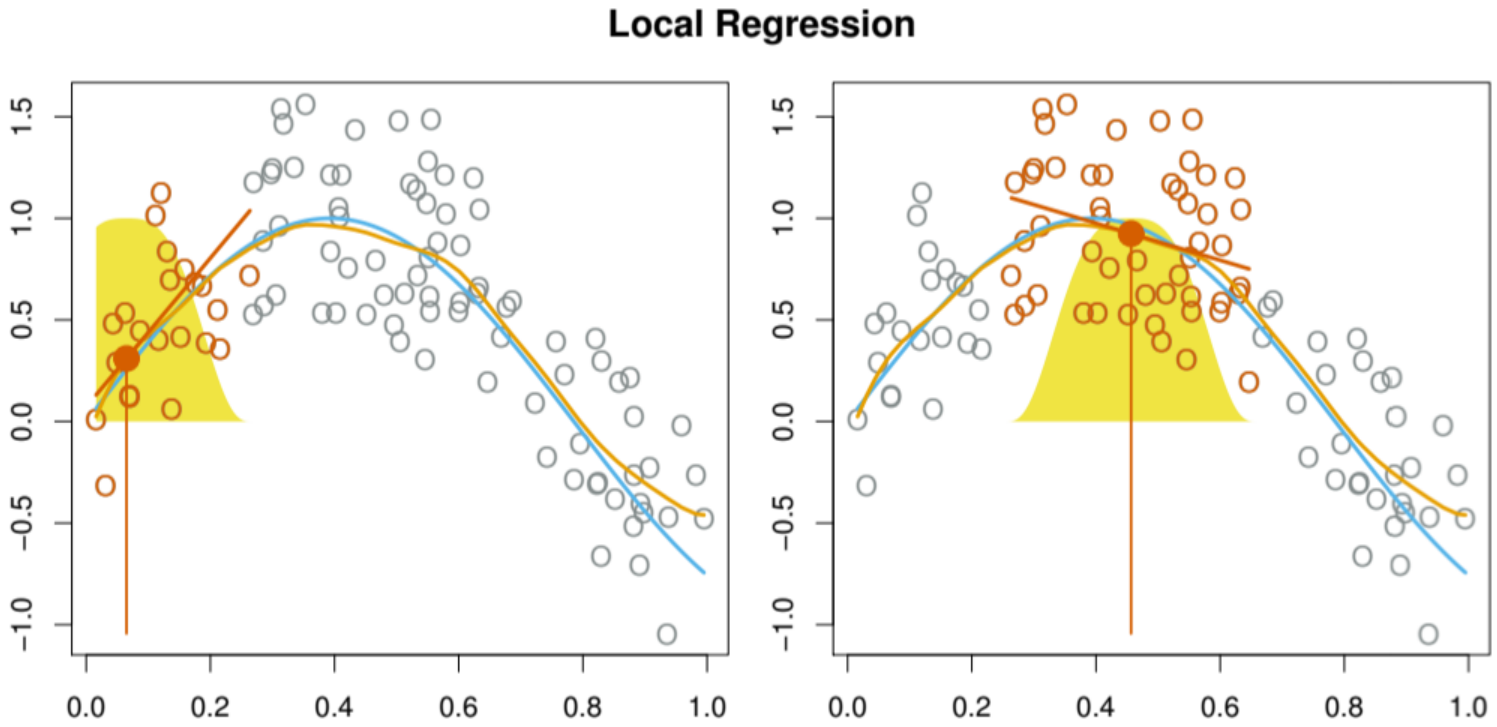
Recall: Locally Weighted Regression

k-nearest neighbour regression

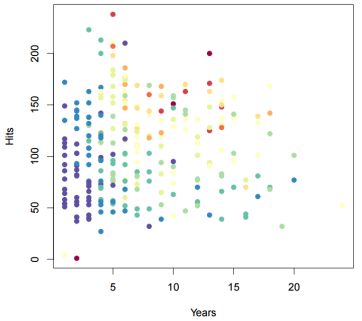
- Locally fit linear model
- Weighted least-squares regression

$$\min \frac{1}{N} \sum_{i=1}^{N} K\left(x^{(i)}, x^{0}\right)\left(\hat{y}^{(i)} - y^{(i)}\right)^{2}$$

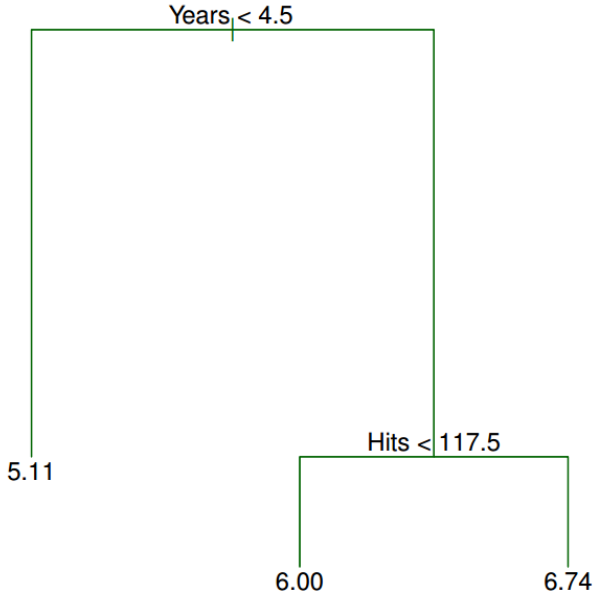
  - E.g. $K(x^{(i)},x^{0})$ are Gaussianweights based on distance between $x^{(i)}$ and x^{0}

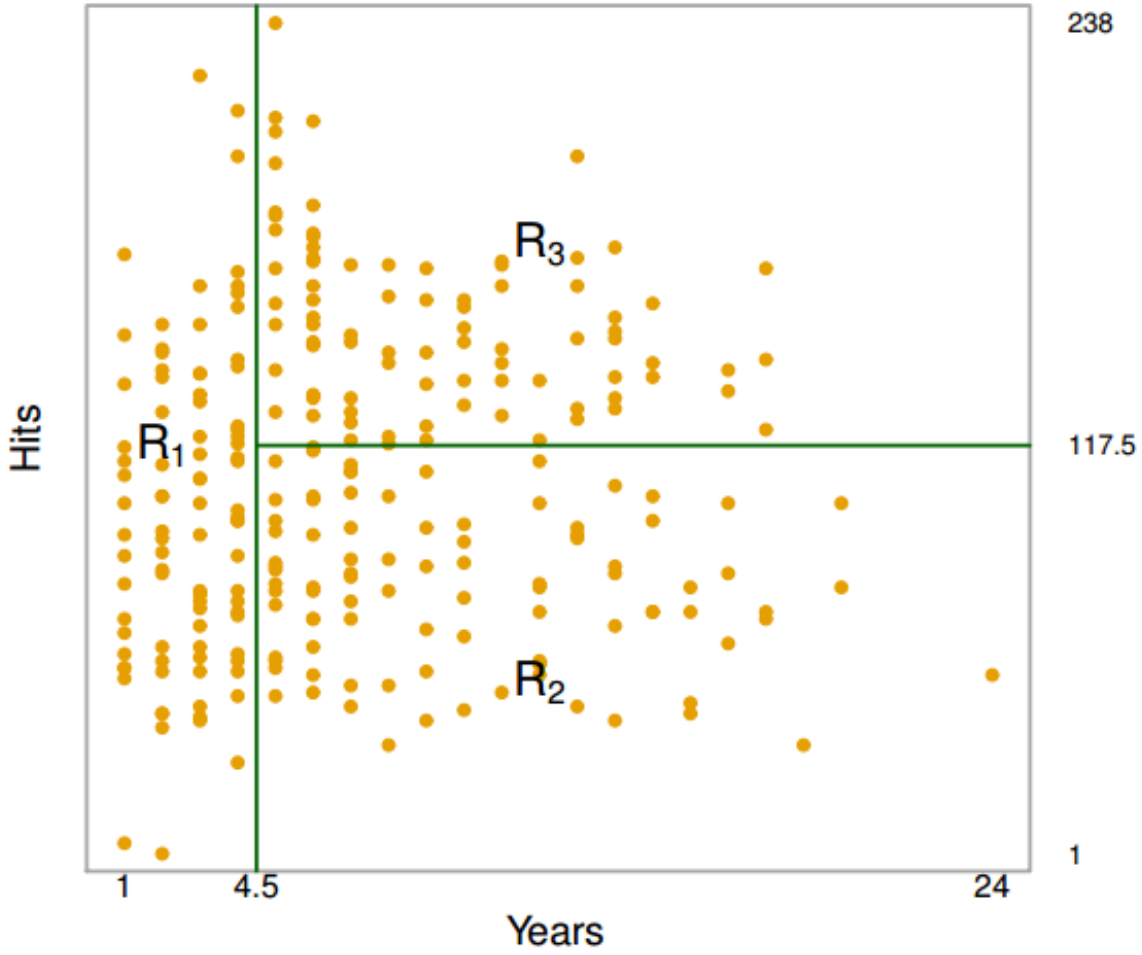


Decision Tree for Regression

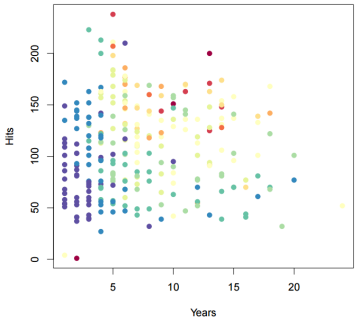


- Overall, the tree partitions the instances into three regions of feature space:
  - R1 ={X | Years< 4.5},
  - R2 ={X | Years>=4.5, Hits<117.5},
  - R3 ={X | Years>=4.5, Hits>=117.5}





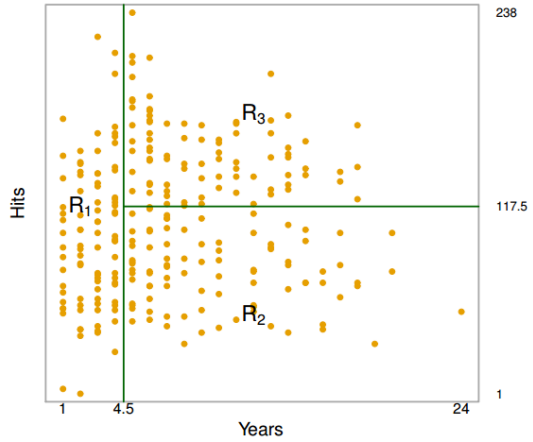
Decision Tree for Regression

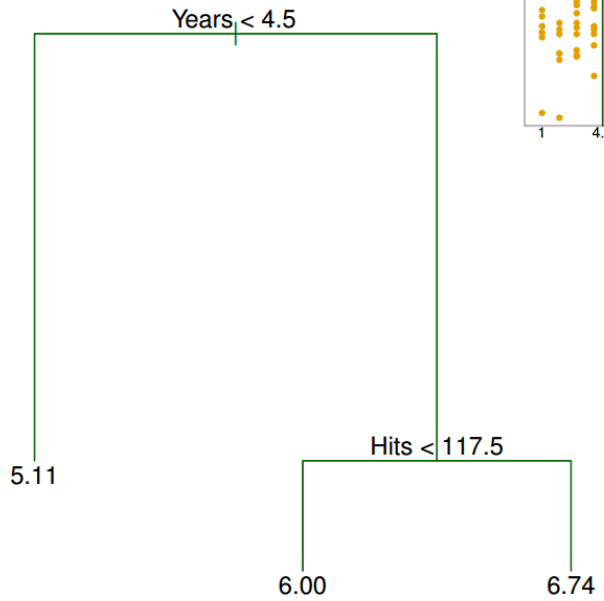


- Overall, the tree partitions the instances into three regions of feature space:
  - R1 ={X | Years< 4.5},
  - R2 ={X | Years>=4.5, Hits<117.5},
  - R3 ={X | Years>=4.5, Hits>=117.5}
- Minimize the following error

$$\sum_{k=1}^{K} \sum_{i \in R_k} \left(y^{(i)} - \hat{y}_{R_k}\right)^2$$

where $\hat{y}_{R_k}$ is the mean response for the kth region R_k





Decision Tree for Regression

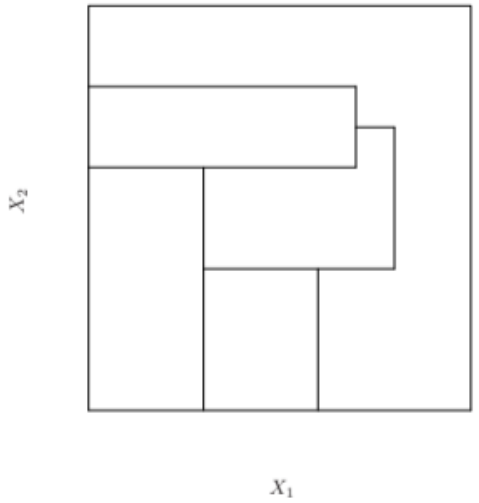
- Take a top-down, greedy, recursive splitting approach
  - At each node of the tree, pick j, s

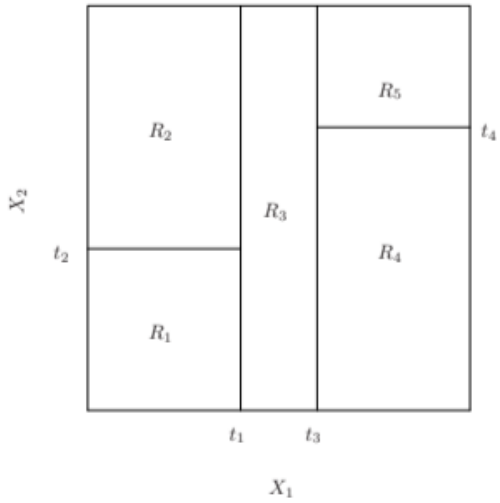
$$R_1(j,s) = \{X \mid X_j < s\} \text{ and } R_2(j,s) = \{X \mid X_j \geq s\}$$

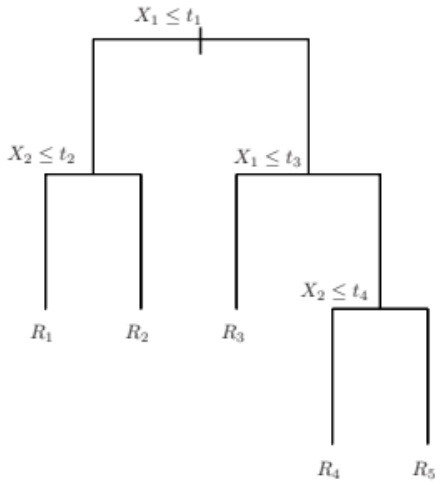
  - that minimizes

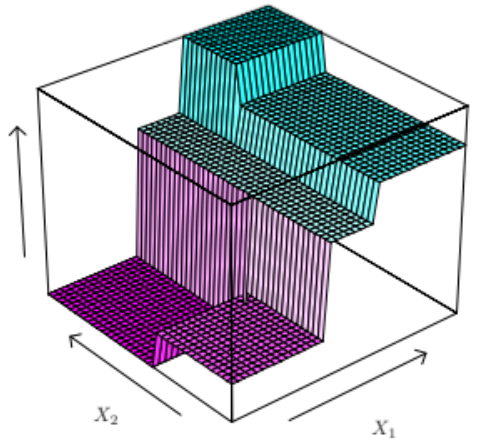
$$\sum_{i:x^{(i)} \in R_1(j,s)} \left(y^{(i)} - \hat{y}_{R_1}\right)^2 + \sum_{i:x^{(i)} \in R_2(j,s)} \left(y^{(i)} - \hat{y}_{R_2}\right)^2$$

where $\hat{y}_{R_k}$ is the mean response for the kth region R_k





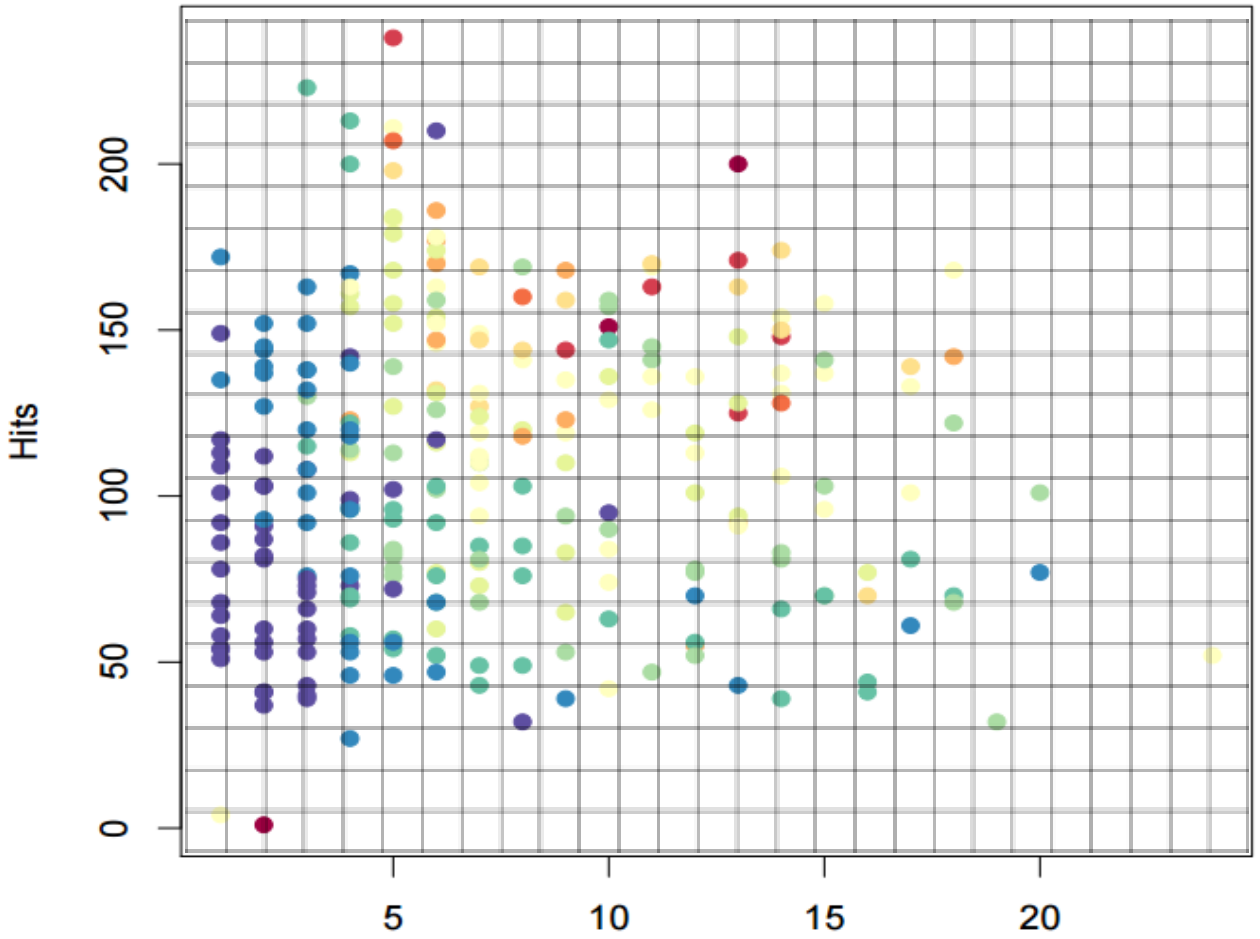




Decision Trees can overfit!

- A DT that is deep enough can create a grid that is fine enough and fits the training data exactly.
- To avoid *overfitting*, regularization is used, by penalizing a highly complex tree, i.e., one that has too many leaf nodes |T|

$$\sum_{k=1}^{|T|} \sum_{x^{(i)} \in R_k} \left(y^{(i)} - \hat{y}_{R_k}\right)^2 + \alpha |T|$$

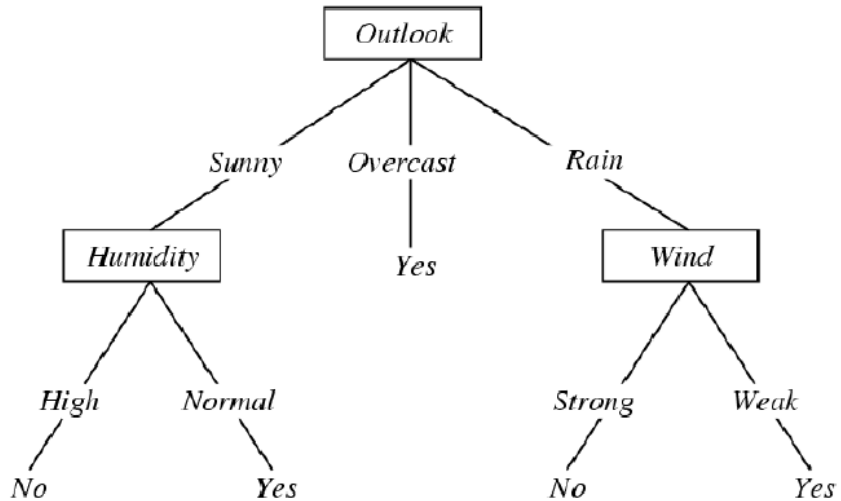


How would we determine α?

Decision Trees for Classification

A Decision tree for

f: <Outlook, Humidity, Wind, Temp> → PlayTennis?



More generally, f: $<X_1, \ldots X_n>$ → Y

Each internal node: discrete test on one attribute, X_i

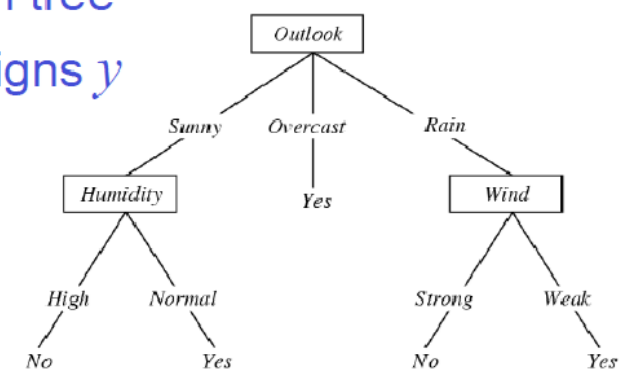
Each branch from a node: selects one value for X_i

Each leaf node: predict Y (or $P(Y|X \in \text{leaf})$)

Decision Tree Learning

Problem Setting:

- Set of possible instances X
  - each instance x in X is a feature vector
  - e.g., <*Humidity=low, Wind=weak, Outlook=rain, Temp=hot*>
- Unknown target function $f : X \rightarrow Y$
  - Y=1 if we play tennis on this day, else 0
- Set of function hypotheses $H=\{ h \mid h : X \rightarrow Y \}$
  - each hypothesis h is a decision tree
  - trees sorts x to leaf, which assigns y



Decision Tree Learning

Problem Setting:

- Set of possible instances X
  - each instance x in X is a feature vector
    $x = < x_1, x_2 \ldots x_n >$
- Unknown target function $f : X \rightarrow Y$
  - Y is discrete-valued
- Set of function hypotheses $H = \{ h \mid h : X \rightarrow Y \}$
  - each hypothesis h is a decision tree

Input:

- Training examples $\{<x^{(i)}, y^{(i)}>\}$ of unknown target function f

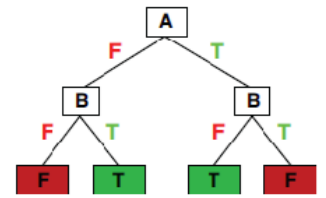
Output:

- Hypothesis $h \in H$ that best approximates target function f

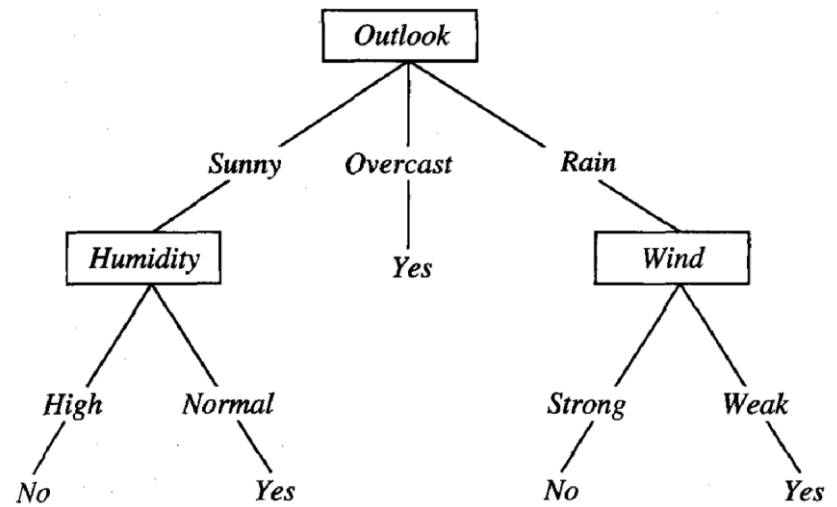
What functions can be represented?

- Decision trees can represent any function of the input attributes!
- For Boolean functions, path to leaf gives truth table row
- But, could require exponentially many nodes…

A	B	A xor B
F	F	F
F	T	T
T	F	T
T	T	F



(Figure from Stuart Russell)



Top-Down Induction of Decision Trees

[ID3, C4.5, Quinlan]

node = Root

Main loop:

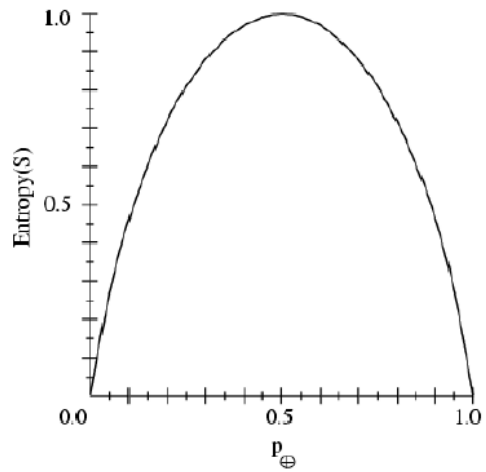
1. $A \leftarrow$ the "best" decision attribute for next *node*
2. Assign A as decision attribute for *node*
3. For each value of A, create new descendant of *node*
4. Sort training examples to leaf nodes
5. If training examples perfectly classified, Then STOP, Else iterate over new leaf nodes

Which attribute is best?

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Sample Entropy



- S is a sample of training examples
- $p_\oplus$ is the proportion of positive examples in S
- $p_\ominus$ is the proportion of negative examples in S
- Entropy measures the impurity of S

$$H(S) \equiv -p_\oplus \log_2 p_\oplus - p_\ominus \log_2 p_\ominus$$

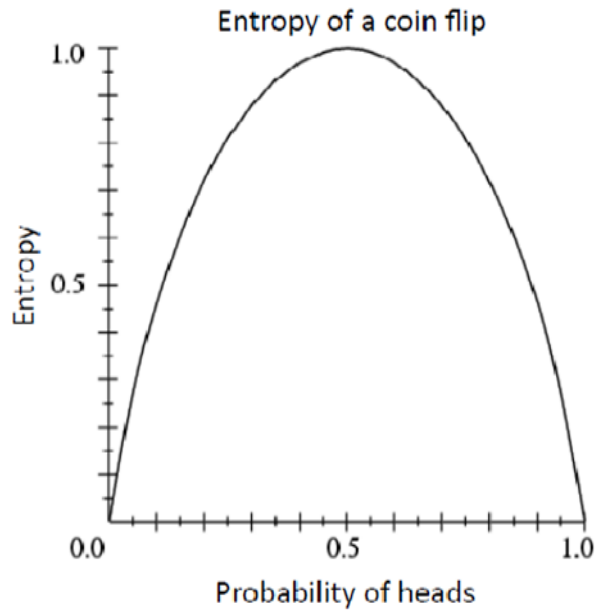
Entropy

Entropy $H(Y)$ of a random variable Y

$$H(Y) = -\sum_{i=1}^{k} P(Y = y_i) \log_2 P(Y = y_i)$$

More uncertainty, more entropy!

Information Theory interpretation: $H(Y)$ is the expected number of bits needed to encode a randomly drawn value of Y (under most efficient code)



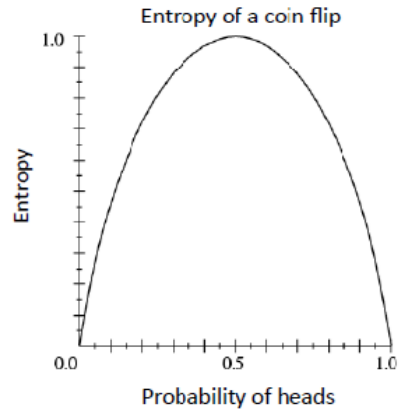
High, Low Entropy

- "High Entropy"
  - X is from a uniform like distribution
  - Flat histogram
  - Values sampled from it are less predictable
- "Low Entropy"
  - X is from a varied (peaks and valleys) distribution
  - Histogram has many lows and highs
  - Values sampled from it are more predictable

(Slide from Vibhav Gogate)

Entropy Example

$$H(Y) = -\sum_{i=1}^{k} P(Y = y_i) \log_2 P(Y = y_i)$$



P(Y=t) = 5/6

P(Y=f) = 1/6

H(Y) = - 5/6 $\log_2$ 5/6 - 1/6 $\log_2$ 1/6

= 0.65

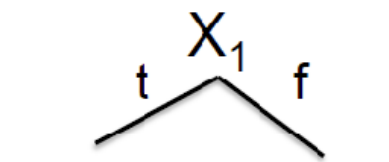
Y
T
T
T
T
T
F

Conditional Entropy

Conditional Entropy $H(Y|X)$ of a random variable Y conditioned on a random variable X

$$H(Y \mid X) = -\sum_{j=1}^{v} P(X = x_j) \sum_{i=1}^{k} P(Y = y_i \mid X = x_j) \log_2 P(Y = y_i \mid X = x_j)$$

Example:



$P(X_1=t) = 4/6$

$P(X_1=f) = 2/6$

Y=t : 4
Y=f : 0

Y=t : 1
Y=f : 1

$H(Y|X_1) = -\ 4/6\ (1 \log_2 1 + 0 \log_2 0)$
$- 2/6\ (1/2 \log_2 1/2 + 1/2 \log_2 1/2)$
$= 2/6$

X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F

Entropy

Entropy $H(X)$ of a random variable X

$$H(X) = -\sum_{i=1}^{n} P(X=i)\log_2 P(X=i)$$

Specific conditional entropy $H(X|Y=v)$ of X given $Y=v$:

$$H(X|Y=v) = -\sum_{i=1}^{n} P(X=i|Y=v)\log_2 P(X=i|Y=v)$$

Conditional entropy $H(X|Y)$ of X given Y :

$$H(X|Y) = \sum_{v \in values(Y)} P(Y=v)H(X|Y=v)$$

Mutual information (aka Information Gain) of X and Y :

$$I(X,Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

Information gain

- Decrease in entropy (uncertainty) after splitting

$$IG(X) = H(Y) - H(Y \mid X)$$

In our running example:

$IG(X_1) = H(Y) - H(Y|X_1)$
$= 0.65 - 0.33$

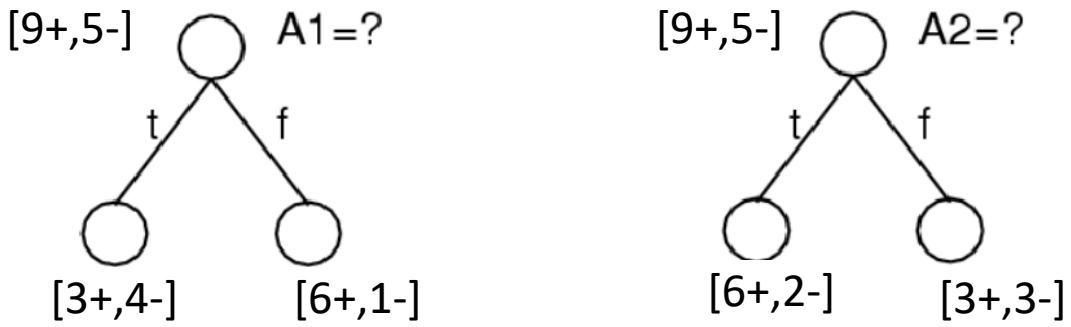
$IG(X_1) > 0$ → we prefer the split!

X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F

Information Gain is the mutual information between input attribute A and target variable Y

Information Gain is the expected reduction in entropy of target variable Y for data sample S, due to sorting on variable A

$$Gain(S, A) = I_S(A, Y) = H_S(Y) - H_S(Y|A)$$



PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

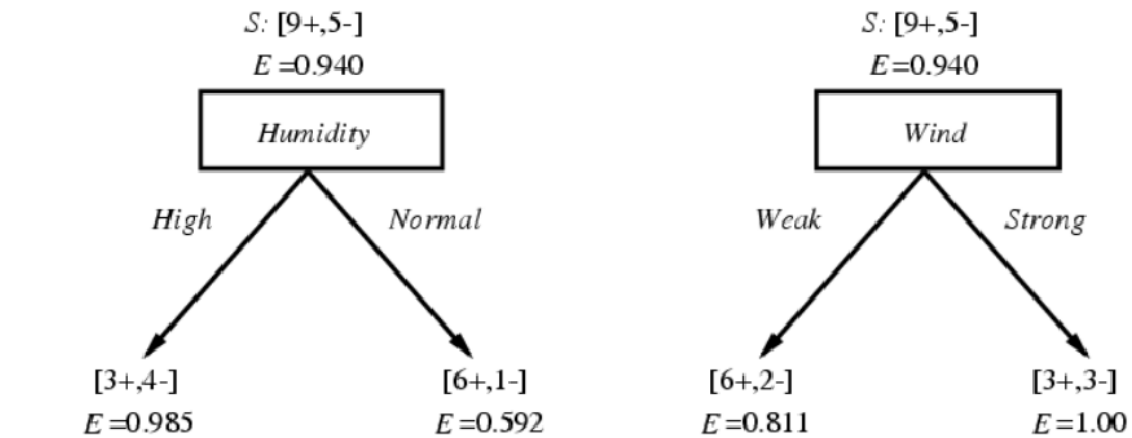
$$H(Y \mid X) = -\sum_{j=1}^{v} P(X = x_j) \sum_{i=1}^{k} P(Y = y_i \mid X = x_j) \log_2 P(Y = y_i \mid X = x_j)$$

$$H(Y) = -\sum_{i=1}^{k} P(Y = y_i) \log_2 P(Y = y_i)$$

$$IG(X) = H(Y) - H(Y \mid X)$$

Selecting the Next Attribute

Which attribute is the best classifier?

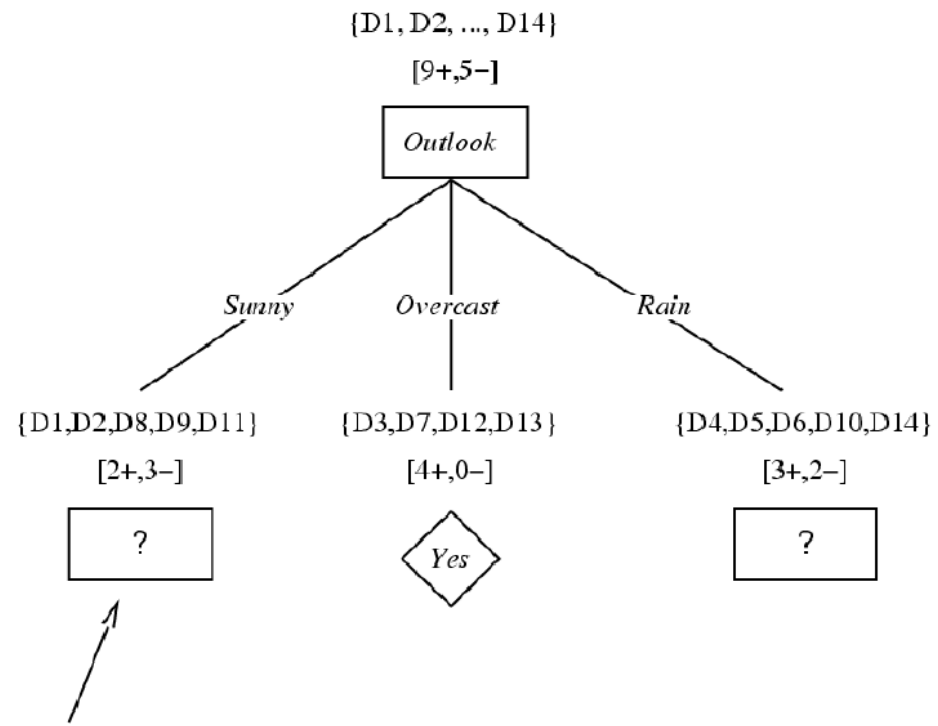


Gain (S, Humidity)
= .940 - (7/14).985 - (7/14).592
= .151

Gain (S, Wind)
= .940 - (8/14).811 - (6/14)1.0
= .048

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



Which attribute should be tested here?

S_{sunny} = {D1,D2,D8,D9,D11}

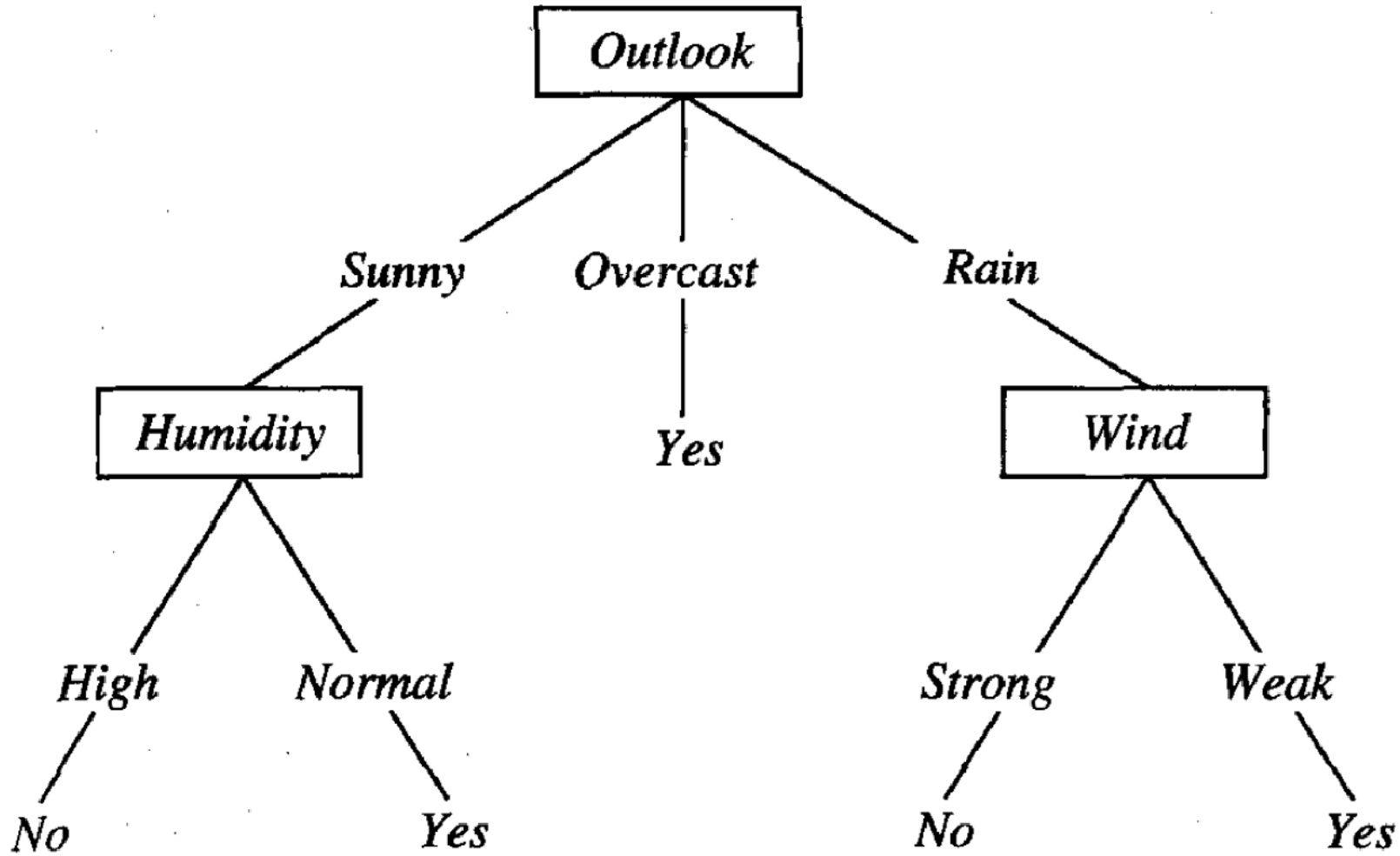
Gain (S_{sunny}, *Humidity*) = .970 − (3/5) 0.0 − (2/5) 0.0 = .970

Gain (S_{sunny}, *Temperature*) = .970 − (2/5) 0.0 − (2/5) 1.0 − (1/5) 0.0 = .570

Gain (S_{sunny}, *Wind*) = .970 − (2/5) 1.0 − (3/5) .918 = .019

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No


Outlook
Sunny
Overcast
Rain
Humidity
Yes
Wind
High
Normal
Strong
Weak
No
Yes
No
Yes

Continuous Valued Attributes

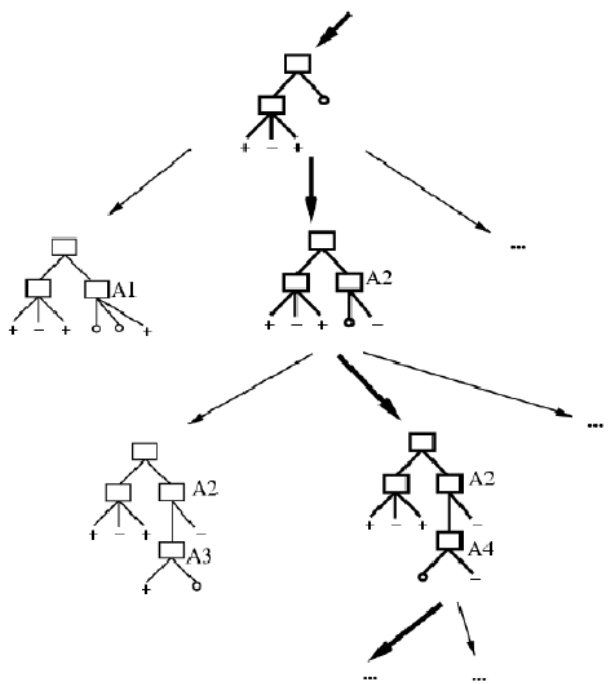
Create a discrete attribute to test continuous

- $Temperature = 82.5$
- $(Temperature > 72.3) = t, f$

Temperature:	40	48	60	72	80	90
PlayTennis:	No	No	Yes	Yes	Yes	No

Jugaad – to deal with continuous variables!

Which Tree Should We Output?



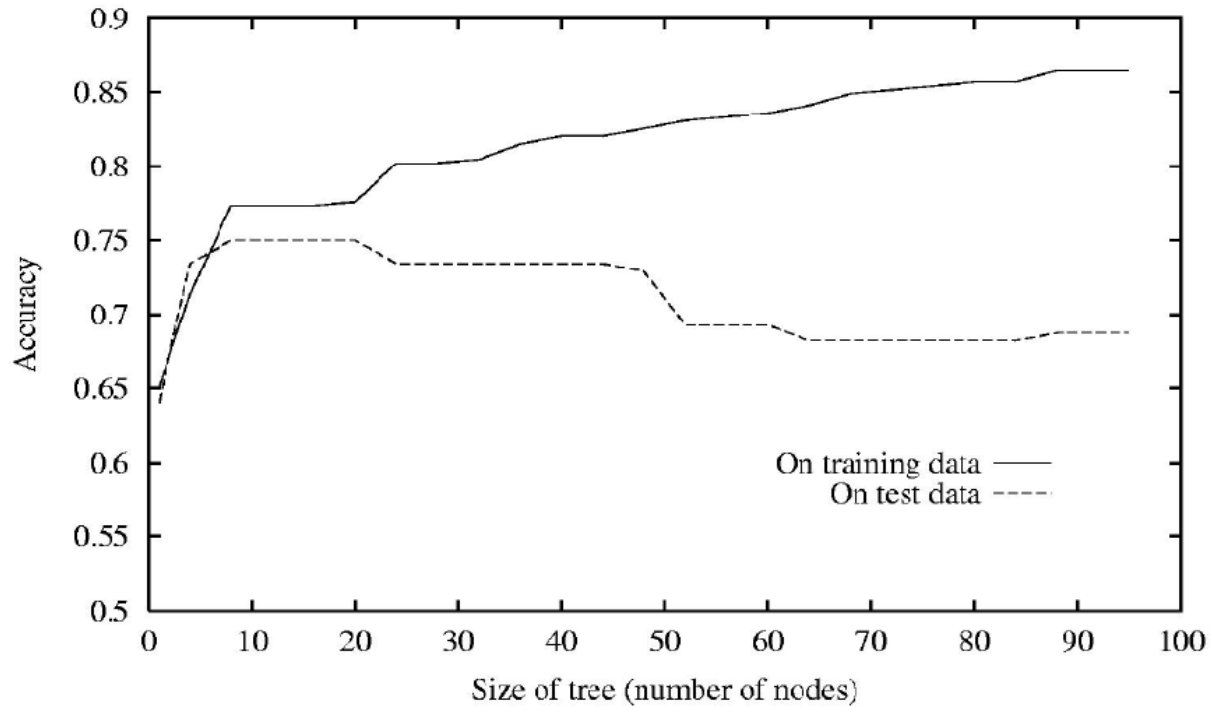
- ID3 performs heuristic search through space of decision trees
- It stops at smallest acceptable tree. Why?

Occam's razor: prefer the simplest hypothesis that fits the data

Decision trees will overfit!!!

- Standard decision trees have no learning bias
  - Training set error is always zero!
    - (If there is no label noise)
  - Lots of variance
  - Must introduce some bias towards simpler trees
- Many strategies for picking simpler trees
  - Fixed depth
  - Fixed number of leaves
  - Or something smarter…

Decision trees will overfit!!!



How to Build Small Trees

Two reasonable approaches:

- Optimize on the held-out (development) set
  - If growing the tree larger hurts performance, then stop growing
  - Requires a larger amount of data…
- Use statistical significance testing
  - Test if the improvement for any split it likely due to noise
  - If so, don't do the split!
  - Can also use this to prune the tree bottom-up

Summary

- Decision Trees are simple interpretable models that can be used for regression and classification
  - They partition the feature space to group samples that belong to a class or predict a similar target value.
- Decision Trees can easily overfit data
- Training algorithms are greedy
  - Use measures like Entropy or Gini Index to identify the next decision variable
- Next Lecture
  - Leveraging Statistics to improve Decision Tree performance and reliability
  - Random Forests & Bootstrap Aggregating (commonly known as Bagging)
  - Boosting & Gradient Boosting Trees